HexFence: FINAL v4, LOCKED FOR IMPLEMENTATION

Locked: Friday 6 Nov 2026, 00:30 IST. **Submissions due:** Saturday 7 Nov, 09:00 IST.

This is the last version. No more architecture reviews. Change it only for bugs you hit while testing.

What v4 changes, from GPT's review:

1. The Guard **rejects a second payee with the same fingerprint**, so removing one payee can never weaken another. This is implemented and tested.
2. The CREATE2 "fresh lookalike" demo is **optional**. The synthetic test pair now comes from a **birthday search**, which finds a matching pair in a minute or two. A ready-made pair is in §3.5.
3. **Real evidence and the synthetic test are separate.** The client never says "verified poisoning evidence" for the synthetic address; it says "resembles an approved payee".
4. **Feature freeze.** Only P0 is required. Receipt verification lives in the **explorer only** (P1). Permissionless publishers, privacy buckets, strict mode, approval-spender checks, GitHub Actions and Slither are P2.

Attached: `11_HexFence_Guard_reference/`. This is a reference `HexFenceGuard` with **13 passing tests against the real Safe 1.4.1 contracts**, including the removal drill through a real Safe transaction.

---

0. Lock card

Architecture	Detector → Arkiv → client (Chrome extension **or** HexFence Pay) → Safe Guard
Key innovation	**Deny by resemblance, without needing a previously reported scam address.** The Safe refuses a lookalike of an approved supplier, and a genuinely new supplier still goes through.
Sponsors	**Arkiv**: Build on Arkiv (from scratch). **Ethereum Foundation**: Best CROPS App, plus Best Use of Anon-RPC only if P1 Anon-RPC is done; EF places you in one. **No Chainlink.**
Non-negotiable demo	① lookalike **rejected** on-chain · ② legitimate recipients **allowed** (the approved payee **and** a new supplier) · ③ Guard **removal tested** through a real Safe transaction · ④ real mainnet evidence visible from Arkiv
Owners	**Baqir**: Guard (start from the reference) + Sepolia Safe + removal drill · **Dhruv**: client (extension, or HexFence Pay) · **Deven**: detector + Arkiv writes · **Akshat**: explorer + README/CROPS/Arkiv files + video
01:00 IST decision	Can Dhruv capture and decode Safe's signing request reliably? **Yes →** extension. **No →** HexFence Pay. **Don't extend the deadline.**
Gate (Fri 14:00)	①–④ work end to end on Sepolia + Arkiv. **Nothing from P1 starts before this.**
Cut rule	If it doesn't directly establish ①–④, it waits or gets cut.

---

1. Scope (the hard list)

P0: must work by the Fri 14:00 gate

1. **Guard:** the reference `HexFenceGuard` deployed on Sepolia, set on an **isolated demo Safe**, with one approved payee. The removal drill is done in Safe{Wallet} and both transaction hashes are recorded.
2. **Detector → Arkiv:** real mainnet USDT/USDC poisoning pairs (**≥5 at the gate**, ≥20 by submission) written to Arkiv. Each record carries transaction hashes and is idempotent (re-run → 0 duplicates).
3. **Explorer:** reads Arkiv and lets you search by address. It shows the victim, the impersonated address and the lookalike, with Etherscan links for the attack and reference transactions.
4. **Client:** the extension (Plan A) or HexFence Pay (Plan B) decodes the payment and reads `payees(safe)` from the Guard. It shows the **resemblance** warning and makes you type 6 characters to continue anyway.
5. **Docs:** README (incl. the **CROPS** section and limits), `/arkiv/*.md` (the five files), `docs/AI_USAGE.md`, and `docs/planning/` with all plans, prompts and reports.

P1: only after the gate, in this order

1. Explorer **"Verify on Ethereum"**, which re-checks the receipts in the browser (Akshat).
2. Client **"listed in public evidence"** signal. It reads the Arkiv records in bulk and fires only on an **exact** address match (Dhruv).
3. **Anon-RPC** for the client's RPC reads, by default. This is needed only to compete for EF's Anon-RPC track.
4. **Hindi** text in the verdict.
5. ≥20 pairs plus a precision check on 30 hand-labelled samples (Deven).

P2: only if everything above is done (expect to cut)

- strict mode
- approval-spender checks
- GitHub Actions publisher
- Slither
- multi-publisher config
- k-anonymous buckets
- CREATE2 lookalike of a *given* address
- multi-window fingerprints
- Marathi
- MultiSend
- Telegram / CRE

---

2. Hard-coded parameters

Item	Value
Enforcement chain	**Sepolia** (chain ID 11155111)

Item	Value
Evidence source	**Ethereum mainnet** (chain ID 1)
Arkiv network	**Rabdi testnet**, chain ID **7794095**, RPC `https://rpc.rabdi.db-chain.testnet.arkiv.network`. Faucet and access keys at `hub.arkiv.network`.
Mainnet tokens	USDT `0xdAC17F958D2ee523a2206206994597C13D831ec7`, USDC `0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48` (both 6 decimals)
Transfer topic0	`0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef`
Fingerprint	`fp(a) = (uint160(a) >> 144) << 16 | (uint160(a) & 0xFFFF)` as `bytes4`. That is literally **first 4 hex ‖ last 4 hex**, so `0x2ef9…8bf7` → `0x2ef98bf7`. The TypeScript and Solidity versions must be identical.
Fingerprint test vectors	USDT → `0xdac11ec7` · USDC → `0xa0b8eb48` · `0x000…000` → `0x00000000` · `0x2ef9a83c5e1b77d04c3f9a2b6d81e0c452d18bf7` → `0x2ef98bf7` · `0x2ef970bc13aa94e2f05c7d6b8e3a21f991f38bf7` → `0x2ef98bf7`
Detector range	The latest **7,200 blocks** (about 24 hours), with `eth_getLogs` in **100-block windows**. Lower the window if your RPC limits responses.
Reference payment	`value ≥ 1,000,000` (≥ 1 token)
Zero-value variant	`Transfer(from = victim, to = lookalike, value = 0)`
Dust variant	`Transfer(from = lookalike, to = victim, 0 < value < 1,000,000)`
Match rule	lookalike ≠ reference recipient, `fp(lookalike) == fp(reference recipient)`, and the reference comes **before** the attack (block, then log index), within 7,200 blocks
Arkiv schema	`hexfence.poisoning.v1` (fields in §3.2), expiry **30 days**
Status label	Always "**suspected**"; never "malicious" or "scammer"
Guard errors	`LookalikeOfApprovedPayee(recipient, payee)`, `FingerprintAlreadyApproved(fp, existing)`, `NotApprovedPayee(payee)`, `DelegateCallNotAllowed(target)`, `ZeroAddressPayee()`
Safe interface	Guard interface ID `0xe6d7a83a`; guard storage slot `0x4a204f62…c34c8`; event `ChangedGuard`
Continue anyway on RED	Type the **last 6 hex characters** of the recipient shown
Measured gas	A guarded ETH payment cost **~68k gas** for the whole `execTransaction` (reference test)

3. Components

3.1 Guard (Baqir): start from `11_HexFence_Guard_reference/`

Storage (GPT fix #1, chosen design: reject duplicates):

- `payeeByFingerprint[safe][fp] → the one approved payee`;
- `_payees[safe]` (an enumerable list);

- `_indexPlusOne[safe][payee]`.

`approvePayee` reverts `FingerprintAlreadyApproved` if another approved payee already holds that fingerprint. So `removePayee` only ever clears its own fingerprint and can't touch another payee's protection.

Why reject rather than support several payees per fingerprint?

- Two *legitimate* payees collide only about 1 in 4.3 billion times.
- A collision during approval is itself a **poisoning signal**: you may be about to approve the lookalike.
- The code stays simpler.

`checkTransaction`, in this order:

1. `operation == Call && (to == safe || to == guard)` → return. This is the administration and recovery path, and it runs before any decoding.
2. `DelegateCall` → revert `DelegateCallNotAllowed` (fail closed).
3. Decode:
   - empty data → `to`;
   - `transfer` (`0xa9059cbb`) → argument 0;
   - `transferFrom` (`0x23b872dd`) → argument 1;
   - anything else → return without inspecting. Clients show "cannot verify".
4. The recipient is approved → allow.
5. Its fingerprint is held by an approved payee → revert `LookalikeOfApprovedPayee`.
6. Otherwise → allow, because this is a genuinely new recipient.

Tests (already passing in the reference, 13/13):

- interface + installation;
- fingerprint vectors;
- an approved payee is allowed;
- a lookalike is refused (ETH, `transfer`, `transferFrom`);
- a new recipient is allowed;
- delegatecall is refused;
- a duplicate fingerprint is rejected;
- removing one payee leaves the others protected;
- each caller's configuration is separate;
- the administration path and junk calldata never revert;
- **`setGuard(0)` through a real Safe transaction works while the guard is active, and re-enabling restores protection**;
- gas.

Live drill (P0): remove the guard through **Safe{Wallet}** on Sepolia, re-enable it, and record both transaction hashes.

Claims we don't make:

- recovery is guaranteed (Safe warns a guard can lock a Safe; the demo Safe is isolated and holds test funds only);
- module transactions or MultiSend are protected;
- unknown calls are inspected;
- weaker (fewer than 4+4) lookalikes are caught.

The reference tests use **Hardhat** because they run the real Safe contracts in-process with no RPC. Keep them, or port them to Foundry only if it's quick.

3.2 Detector + Arkiv (Deven)

- **Stack:** TypeScript (Node 22) + viem + `@arkiv-network/sdk`. It writes `evidence.jsonl` locally *and* to Arkiv.
- **Algorithm:**
  - walk the Transfer logs of both tokens in block and log order;
  - keep `refs[victim] = Map<fp, {recipient, block, tx}>` of reference payments within the window;
  - on a zero-value log (victim → lookalike) or a dust log (lookalike → victim), look up `refs[victim].get(fp(lookalike))`;
  - if that recipient is present and isn't the lookalike, you have a match. Emit evidence.
- **Record fields** (types as in the Arkiv docs: `addr`, `str`, `dec`):
  - *indexed:* `evidenceId` (str), `chainId` (dec), `lookalike` (addr), `impersonated` (addr), `victim` (addr), `variant` (str: `zero-value` | `dust`), `attackBlock` (dec), `detectorVersion` (str);
  - *plain:* `token` (addr), `attackTx` (str), `referenceTx` (str), `referenceBlock` (dec), `prefixLen` (dec), `suffixLen` (dec), `status` (str, always "suspected").
  - No made-up "confidence" number.
- **Idempotency:** `evidenceId = keccak256(chainId, lookalike, impersonated, detectorVersion)`. Before creating, query `where(eq('evidenceId', …))` with `createdBy(publisher)`.
- **SDK use** (from the Arkiv docs):
  - `createEntity(Schema, values, { expiration: ExpirationTime.fromDays(30) });`
  - `select(...).from(Schema).where(...).createdBy(addr).limit(n).fetch();`
  - pages hold at most 200 entities, so loop on `hasNextPage()`.
- **README:** the publisher wallet and **one query judges can run**.

3.3 Explorer (Akshat)

- **P0:** a static page (Vite, no backend) that reads Arkiv directly.
  - Search by address: lookalike, impersonated or victim.
  - Each record links its attack and reference transactions on Etherscan.
  - Counts by variant.
  - It's labelled "Ethereum mainnet evidence · status: suspected".
- **P1:** a **"Verify on Ethereum"** button. It fetches both receipts and checks for a `Transfer` log with the canonical token, the right from/to, the variant's value rule, and the reference block being earlier. It shows ✓ or "could not verify". Receipt verification lives **only here**, not in the client.

3.4 Client (Dhruv): extension (Plan A) or HexFence Pay (Plan B)

- **Plan A (Chrome MV3, `app.safe.global`, Sepolia):** a MAIN-world script at `document_start` wraps `window.ethereum` **and** EIP-6963 providers. It intercepts:
  - `eth_signTypedData_v4` with `primaryType === "SafeTx"`;
  - `eth_sendTransaction` to `execTransaction` (`0x6a761202`).

  It decodes the same shapes as the Guard. The verdict opens in its own window (`chrome.windows.create`).
- **Plan B (HexFence Pay):** a static web app that builds the Safe payment itself with Safe's Protocol Kit. It runs the same checks **before** asking the wallet to sign.
- **Reads (P0):** a single `payees(safe)` call to the Guard on Sepolia. **No per-recipient lookup anywhere.**

Verdict signals: two kinds, never mixed (GPT fix #3). The English strings are fixed; the Hindi lines are P1.

Signal	When	Text (English, fixed)
RED: resembles (P0)	The fingerprint matches an approved payee and the address isn't that payee	"**Possible address poisoning.** This recipient resembles your approved payee {payee} (same first 4 and last 4 characters) but is a different address. Your protected Safe will refuse this payment."
RED: listed (P1)	The **exact** address is in the Arkiv evidence records	"This exact address is listed in public poisoning evidence on Arkiv (Ethereum mainnet, publisher {publisher}). Status: suspected. See the explorer."
AMBER: new (P0)	Not an approved payee and no fingerprint match	"New recipient: not one of your approved payees. HexFence will not block this payment. Check the full address with the recipient."
GREEN: approved (P0)	An approved payee	"Approved payee {payee}."
GREY: cannot verify (P0)	Delegatecall or any unsupported calldata	Delegatecall: "This is a delegatecall. Your HexFence Guard will refuse it." Anything else: "Cannot verify this transaction type. HexFence checks only ETH and ERC-20 transfers."

Hindi (P1), have a native speaker check it:

- **RED: resembles:** "संभावित एड्रेस पॉइज़निंग: यह प्राप्तकर्ता आपके स्वीकृत प्राप्तकर्ता {payee} जैसा दिखता है (पहले 4 और आख़िरी 4 अक्षर समान), लेकिन यह अलग पता है। आपका सुरक्षित Safe यह भुगतान नहीं करेगा।"
- **AMBER: new:** "नया प्राप्तकर्ता: यह आपके स्वीकृत प्राप्तकर्ताओं में नहीं है। HexFence इस भुगतान को नहीं रोकेगा। पूरा पता प्राप्तकर्ता से मिलाकर देखें।"

Rules:

- The **synthetic Sepolia lookalike only ever gets "RED: resembles"**, because it isn't in the evidence. Never show "verified" or "evidence" for it.
- The client never uses the words "safe", "verified", "scam" or "malicious".
- RED requires typing the last 6 characters to continue. Reject → EIP-1193 error `4001`.

3.5 Synthetic Sepolia test pair (P0 demo utility, GPT fix #2)

`11_HexFence_Guard_reference/scripts/make-lookalike-pair.js`:

- generates random keys until **two** addresses share a fingerprint;
- by the birthday bound that takes ~**77,000** keys on average (99% by ~199,000);
- we measured ~1,700 keys/s with ethers in Node, so **usually 1–2 minutes**; one test run found a pair after 172,752 keys in 102 s;
- outputs **P** (the approved payee) and **L** (its lookalike), and keeps the keys in git-ignored `.pair.local.json`.

Ready-made pair from that test run. The keys weren't kept, so use these **only as recipients** on Sepolia:

- fingerprint `0x4e184120`
- **P** = `0x4E185e251Ac7fB83E9DE10723EAda46cadE74120`
- **L** = `0x4E185a3FC4B530244aB9ed0dE38167b403344120`

B (a new supplier) is any fresh test address.

Optional (P2): `cast create2 --starts-with <first4> --ends-with <last4>` makes a lookalike of a **given** address. That needs about 4.3 billion attempts on average, so it must never be on the

critical path.

---

4. Demo (two separate parts, clearly labelled)

Part 1: real evidence (Ethereum mainnet → Arkiv). About 1 minute.

1. Open the explorer, which reads Arkiv live.
2. Show the count of suspected poisonings the detector found in the last 24 hours.
3. Open one record: victim, impersonated address, lookalike, and both transactions on Etherscan.
4. *(P1)* Click **Verify on Ethereum** → ✓.

Part 2: synthetic re-enactment (Sepolia). About 2 minutes. Say: "*This is a synthetic test pair, not a real attack.*"

1. The demo Safe has the HexFence Guard and approved payee **P**.
2. Pay **L** → the client shows **RED: resembles P** → type 6 characters to continue anyway → execute → **reverts with** `LookalikeOfApprovedPayee(L, P)` on Sepolia Etherscan.
3. Pay **B** (a new supplier) → AMBER → **succeeds**. "*Not an allowlist: new suppliers still work.*"
4. Pay **P** → GREEN → **succeeds**.
5. *(EF moment)* Switch the client off → pay L again → **still reverts**. "*Enforcement doesn't need our software.*"
6. Show the removal drill transaction hashes: `setGuard(0)` → re-enabled.

Part 3: why it's new. About 1 minute. Wallets warn (MetaMask, Trust Wallet, Safe{Wallet}). Allowlists block new suppliers. Blocklists need someone to have reported the attacker first. HexFence denies by resemblance, at the account level. Then state the limits: modules, MultiSend, weaker lookalikes.

Video: 2:30–3:00, ≥720p. That satisfies both ETHGlobal (2–4 minutes) and EF CROPS (≤3 minutes).

---

5. Innovation and pitch (final wording)

Pitch:

> "HexFence detects address poisoning, publishes the evidence openly on Arkiv, warns you locally before you sign, and makes a protected Safe refuse to pay a lookalike of an approved supplier, even one nobody has ever reported, while new suppliers still work."

	Wallet warnings (MetaMask Jun 2026, Trust Wallet Mar 2026, Safe{Wallet})	**Allowlist guards (Safe Policy Engine, SafeGuard ETHIndia 2022)**	**Blocklists (Safe Policies, vendor feeds)**	**HexFence**
Enforced by the account	✗ (UI, can be overridden)	✓	✓	✓
Stops a never-reported lookalike	✓ in-app only	✓	✗	✓

	Wallet warnings (MetaMask Jun 2026, Trust Wallet Mar 2026, Safe{Wallet})	Allowlist guards (Safe Policy Engine, SafeGuard ETHIndia 2022)	Blocklists (Safe Policies, vendor feeds)	HexFence
New suppliers still work	✓	✗	✓	✓
Open evidence anyone can query	✗	n/a	✗ (closed)	✓ **(Arkiv)**

Honest notes:

- The rule itself is simple ("incremental", as GPT's review put it). The combination is what's new.
- An ETHGlobal showcase search for "poisoning", "lookalike" and "spoof" found no projects. The search covers names and taglines only.
- Never claim "the first recipient guard for Safe".

---

6. CROPS (README section; keep it true to what you ship)

- **C:** The Guard is non-upgradeable and has no owner. It keeps refusing lookalikes even if our detector, explorer, client and Arkiv records all disappear. The detector is open source: anyone can run it and publish under their own wallet.
- **O:** MIT licence. Everything builds and runs from the repo, and RPC endpoints are configurable.
- **P:** The client checks recipients locally, without sending payment-specific threat lookups to a security provider. It reads the Safe's payees in bulk. *(If P1 is done: RPC reads go through Anon-RPC by default.)* There are no trackers.
  - **Still visible:** your wallet, Safe's transaction service (when you use the Safe UI) and the public chain all see the payment.
- **S (lead property):**
  - the account enforces the rule;
  - payees are approved only by owner-signed Safe transactions;
  - duplicate fingerprints are rejected;
  - delegatecall fails closed;
  - removal is tested through real Safe transactions, but not guaranteed;
  - the demo Safe is isolated and holds test funds only;
  - the limits are stated.

---

7. Timeline (from Fri 00:30 IST)

When	Baqir	Dhruv	Deven	Akshat
00:30–01:00	Pull the reference into the repo; `npm i --legacy-peer-deps && npx hardhat test` → 13 passing	**Spike:** capture + decode a real Safe signing request	First 500 blocks; **first Arkiv write + read**	Repo, MIT licence, `docs/planning/`, skeletons for the README and `/arkiv/*.md`
01:00		**Decide Plan A or B**		

When	Baqir	Dhruv	Deven	Akshat
01:00–05:00	Deploy to Sepolia; demo Safe; `setGuard`; approve P; **live removal drill**; record hashes	Client core: decode + `payees(safe)` + verdict + 6-character friction	The 24-hour run → **≥5 pairs** in Arkiv; idempotent re-run	Explorer: search + Etherscan links + counts
05:00–09:00	Sleep in two shifts			
09:00–14:00	Make the synthetic pair (§3.5); end-to-end on Sepolia with Dhruv	Integration with the Guard; fix the decoding edge cases	Fixes; more pairs	The explorer reads the live data; draft the README/CROPS/Arkiv files
14:00	**GATE: ①–④ end to end**			
14:00–20:00	README technical part, limits, tx hashes	P1-2 listed signal → P1-3 Anon-RPC → P1-4 Hindi	P1-5: ≥20 pairs + precision check on 30 labels	P1-1 **Verify on Ethereum**; video script
20:00–02:00	**Three full rehearsals**, fixes, record the video, finish the docs			
02:00–06:00	Freeze, regression run, sleep			
06:00–08:30	**Submit by 08:30.** Partners: **Arkiv + EF**			

8. Submission checklist

- [ ] A public repo created after kickoff, MIT licence, small commits from all four accounts. **No Pramaan code** (it was built before the event and isn't eligible).
- [ ] `docs/planning/`: this file plus every GPT and Claude plan, prompt and report. `docs/AI_USAGE.md` names the reference Guard as AI-generated and reviewed.
- [ ] README:
  - [ ] run steps;
  - [ ] architecture;
  - [ ] what's new vs reused;
  - [ ] Sepolia Guard + Safe addresses;
  - [ ] transaction hashes (approve, lookalike revert, new-supplier success, payee success, removal + re-enable);
  - [ ] Arkiv publisher wallet + one query;
  - [ ] **CROPS** section;
  - [ ] limits.
- [ ] `/arkiv/schema.md`, `friction.md`, `reach.md`, `why-arkiv.md`, `how-you-use-arkiv.md`
- [ ] Video 2:30–3:00, ≥720p

- [ ] Live links: the explorer (static hosting) + extension install steps or the HexFence Pay URL
- [ ] No trackers; testnet keys only; never commit keys or seed phrases

9. Agent prompt (paste into Claude Code or Codex in the NEW repo)

```
Build HexFence for ETHGlobal Mumbai 2026 (Classic: code written now; humans review every change; log
AI use in
docs/AI_USAGE.md; commit small). Read docs/planning/10_FINAL_What_To_Build_HexFence.md (v4, LOCKED)
first.
SCOPE IS LOCKED: build P0 only until the Fri 14:00 gate passes; then P1 in the listed order; P2 only
if everything is done.

P0 FLOW: Detector -> Arkiv -> client (extension OR HexFence Pay) -> HexFenceGuard on Sepolia.
Must demo: lookalike rejected on-chain; approved payee AND a new supplier allowed; guard removal
through a real Safe tx;
real mainnet evidence visible from Arkiv.

RULES
- Nothing fake or hard-coded. Unsupported => "cannot verify". Evidence status is always "suspected".
- Real evidence (mainnet, Arkiv) and the synthetic Sepolia pair are separate: the synthetic lookalike
only ever gets
  "resembles your approved payee", never "verified" or "evidence". No per-recipient lookups anywhere.
- Read official docs before using an API (Safe guards + safe-deployments, Arkiv SDK:
defineSchema/createEntity/
  select-from-where(eq)/createdBy/limit/fetch with pages <= 200, Safe Protocol Kit for Plan B, Anon-
RPC for P1).
  If docs disagree with the spec, stop and ask.
- Testnet keys only (.env, git-ignored). Never print or commit keys or seed phrases.

GUARD: start from 11_HexFence_Guard_reference (Hardhat + real Safe 1.4.1, 13 tests). Keep: duplicate-
fingerprint
rejection, admin path first, delegatecall fail-closed, ETH/transfer/transferFrom decoding,
LookalikeOfApprovedPayee.
Deploy to Sepolia; Safe{Wallet} Transaction Builder, one call per tx: setGuard, approvePayee; removal
drill setGuard(0) then re-enable.

SHARED: packages/shared with fingerprint(addr) = (addr >> 144) << 16 | (addr & 0xFFFF) as bytes4,
using these vectors:
USDT -> 0xdac11ec7, USDC -> 0xa0b8eb48, zero -> 0x00000000; Safe payment decoder (SafeTx typed data +
execTransaction 0x6a761202).

DETECTOR: viem mainnet getLogs (USDT/USDC Transfer, latest 7,200 blocks, 100-block windows);
reference value >= 1e6;
zero-value victim->lookalike; dust lookalike->victim (0 < v < 1e6); match fp(lookalike) ==
fp(reference recipient),
lookalike != recipient, reference earlier. Write evidence.jsonl + Arkiv entities (schema
hexfence.poisoning.v1, fields as
in the spec, 30-day expiry), idempotent by evidenceId = keccak256(chainId, lookalike, impersonated,
detectorVersion).

EXPLORER: static Vite app reading Arkiv: search by address, Etherscan links, counts; P1 "Verify on
Ethereum" (receipts).
CLIENT: Plan A MV3 extension on app.safe.global (MAIN-world wrapper for window.ethereum + EIP-6963;
intercept SafeTx
typed data and execTransaction) or Plan B HexFence Pay (Safe Protocol Kit builds the payment). Read
guard.payees(safe)
once; verdicts RED resembles / AMBER new / GREEN approved / GREY cannot verify, with the exact
English strings in §3.4;
RED needs the last 6 characters typed to continue; reject => error 4001. P1: listed signal (exact
match), Anon-RPC, Hindi.
UTIL: 11_HexFence_Guard_reference/scripts/make-lookalike-pair.js (birthday search for two keys with
the same fingerprint;
```

```
~1-2 min); a ready pair is in spec section 3.5.
After each step: run tests, fix failures, commit, print exact reproduce commands.
```

Appendix: reference `HexFenceGuard.sol` (tested; full repo in `11_HexFence_Guard_reference/`)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {Enum} from "@safe-global/safe-contracts/contracts/common/Enum.sol";
import {BaseGuard} from "@safe-global/safe-contracts/contracts/base/GuardManager.sol";

/// Deny by resemblance: refuse ETH / ERC-20 payments to an address sharing the first 4 + last 4 hex
chars
/// of an approved payee but not equal to it. New recipients are allowed. Non-upgradeable, no owner.
contract HexFenceGuard is BaseGuard {
    error LookalikeOfApprovedPayee(address recipient, address approvedPayee);
    error FingerprintAlreadyApproved(bytes4 fingerprint, address existingPayee);
    error NotApprovedPayee(address payee);
    error DelegateCallNotAllowed(address target);
    error ZeroAddressPayee();

    event PayeeApproved(address indexed safe, address indexed payee, bytes4 fingerprint);
    event PayeeRemoved(address indexed safe, address indexed payee, bytes4 fingerprint);

    mapping(address => mapping(bytes4 => address)) public payeeByFingerprint; // duplicates rejected
    mapping(address => address[]) private _payees;
    mapping(address => mapping(address => uint256)) private _indexPlusOne;   // 0 = not approved

    bytes4 private constant TRANSFER = 0xa9059cbb;
    bytes4 private constant TRANSFER_FROM = 0x23b872dd;

    function fingerprint(address a) public pure returns (bytes4) {
        uint160 x = uint160(a);
        return bytes4(uint32(((x >> 144) << 16) | (x & 0xFFFF)));
    }
    function isApprovedPayee(address safe, address payee) public view returns (bool) {
        return _indexPlusOne[safe][payee] != 0;
    }
    function payees(address safe) external view returns (address[] memory) { return _payees[safe]; }

    function approvePayee(address payee) external {           // called BY the Safe (Safe
transaction)
        if (payee == address(0)) revert ZeroAddressPayee();
        address safe = msg.sender;
        if (_indexPlusOne[safe][payee] != 0) return;
        bytes4 fp = fingerprint(payee);
        address existing = payeeByFingerprint[safe][fp];
        if (existing != address(0)) revert FingerprintAlreadyApproved(fp, existing);
        payeeByFingerprint[safe][fp] = payee;
        _payees[safe].push(payee);
        _indexPlusOne[safe][payee] = _payees[safe].length;
        emit PayeeApproved(safe, payee, fp);
    }

    function removePayee(address payee) external {            // called BY the Safe (Safe
transaction)
        address safe = msg.sender;
        uint256 idx = _indexPlusOne[safe][payee];
        if (idx == 0) revert NotApprovedPayee(payee);
        address[] storage list = _payees[safe];
        address last = list[list.length - 1];
        list[idx - 1] = last;
        _indexPlusOne[safe][last] = idx;
```

```
        list.pop();
        delete _indexPlusOne[safe][payee];
        bytes4 fp = fingerprint(payee);
        delete payeeByFingerprint[safe][fp];                  // safe: only this payee can hold fp
        emit PayeeRemoved(safe, payee, fp);
    }

    function checkTransaction(address to, uint256, bytes memory data, Enum.Operation operation,
        uint256, uint256, uint256, address, address payable, bytes memory, address) external view
override {
        address safe = msg.sender;
        if (operation == Enum.Operation.Call && (to == safe || to == address(this))) return; // admin
+ recovery
        if (operation == Enum.Operation.DelegateCall) revert DelegateCallNotAllowed(to);     // fail
closed
        address recipient;
        if (data.length == 0) recipient = to;
        else if (data.length >= 68 && bytes4(data) == TRANSFER) recipient = _addressAt(data, 4);
        else if (data.length >= 100 && bytes4(data) == TRANSFER_FROM) recipient = _addressAt(data,
36);
        else return;                                                                          // not
inspected
        if (_indexPlusOne[safe][recipient] != 0) return;
        address approved = payeeByFingerprint[safe][fingerprint(recipient)];
        if (approved != address(0)) revert LookalikeOfApprovedPayee(recipient, approved);
    }

    function checkAfterExecution(bytes32, bool) external view override {}

    function _addressAt(bytes memory data, uint256 offset) private pure returns (address a) {
        assembly { a := and(mload(add(add(data, 32), offset)),
0xffffffffffffffffffffffffffffffffffffffff) }
    }
}
```

Sources

- ETHGlobal Mumbai rules
- ETHGlobal Mumbai prizes
- Safe docs: Guard tutorial and lockout warning
- Safe Research: Policy Engine
- Candide: Safe Policies
- SafeGuard (ETHIndia 2022)
- MetaMask: address poisoning detection
- Trust Wallet (The Block)
- USENIX Security 2025: Blockchain Address Poisoning
- The Block: $50M loss
- Arkiv: first entity
- Arkiv query skill
- Foundry `cast create2`
- The reference Guard's tests were run against `@safe-global/safe-contracts@1.4.1-build.0` (npm).